Faculdade de Tecnologia
“José Crespo Gonzales”

# Sistemas Distribuídos Aplicado à Internet das Coisas (IAL023)



# TÓPICOS DA AULA

- 🎯 Algoritmos Distribuídos
- 🔄 **Algoritmos de Comunicação e Consenso**
- Exercícios

## 🧠 Conceito de Algoritmos Distribuídos

Um algoritmo distribuído é um conjunto de regras de execução que se caracteriza por:

**Execução Cooperativa:** Envolve múltiplos processos (programas em execução) ou nós (computadores, dispositivos, etc.) trabalhando juntos.

**Comunicação por Mensagens:** Os processos/nós interagem uns com os outros enviando e recebendo mensagens. Eles não compartilham uma memória central.

# 🧠 Conceito de Algoritmos Distribuídos

**Objetivo Comum:** O objetivo é resolver um problema coletivo, algo que seria difícil ou impossível para um único processo resolver sozinho.

**Controle Não Centralizado:** Esta é a característica chave. Não existe um "chefe" ou um único ponto de controle que dite o que todos devem fazer. Cada nó/processo toma suas decisões com base em seu conhecimento local e nas mensagens que recebe dos outros.

**Em termos simples, é uma maneira de fazer com que vários computadores ou dispositivos trabalhem em conjunto, de forma independente, para atingir um objetivo comum.**

## 🧩 Exemplos Reais Detalhados:

**Blockchain (Bitcoin, Ethereum):** O algoritmo de consenso (como o *Proof-of-Work* ou *Proof-of-Stake*) é distribuído. Milhares de nós (computadores) verificam e validam transações de forma independente, garantindo que o *ledger* (registro de transações) seja o mesmo para todos, sem a necessidade de um banco central.

**Bancos de Dados Distribuídos (Cassandra, MongoDB):** Esses sistemas armazenam partes dos dados em diferentes máquinas. O algoritmo lida com a distribuição das consultas, garante que os dados sejam consistentes (ou eventualmente consistentes) e que o sistema continue funcionando mesmo que uma máquina falhe.

## 🧩 Exemplos Reais Detalhados:

**Aplicações em Nuvem (Google Cloud, AWS):** A infraestrutura da nuvem é massivamente distribuída. Algoritmos gerenciam a alocação de recursos, o balanceamento de carga (distribuindo requisições entre muitos servidores) e a tolerância a falhas.

**Redes de Sensores e Internet das Coisas (IoT):** Muitos dispositivos (sensores de temperatura, câmeras, etc.) coletam dados e se comunicam em rede. Os algoritmos distribuídos ajudam a gerenciar a coleta de dados, o processamento e a coordenação das ações desses dispositivos.

# Propriedades de Algoritmos Distribuídos

| Propriedade | Descrição | Importância |
| --- | --- | --- |
| **Correção** | O algoritmo deve produzir resultados válidos. | **Fundamental.** Garante que o sistema nunca atinja um "estado ruim" e que a resposta final é a correta para o problema proposto. |
| **Terminação** | Todos os processos devem concluir sua execução. | **Importante.** Garante que o sistema eventualmente atinja um "estado bom" (o resultado final). Um algoritmo que nunca termina é inútil. |

# Propriedades de Algoritmos Distribuídos

| Propriedade | Descrição | Importância |
| --- | --- | --- |
| **Convergência** | Todos os nós devem chegar ao mesmo estado final. | **Crítica.** Essencial para o consenso e a consistência de dados. Significa que, apesar de iniciarem de estados diferentes ou receberem informações em momentos diferentes, todos os nós concordarão com o resultado final (por exemplo, qual transação foi aceita no Blockchain). |

# Propriedades de Algoritmos Distribuídos

| Propriedade | Descrição | Importância |
| --- | --- | --- |
| **Equidade** | Nenhum processo é permanentemente excluído ou impedido de progredir. | **Necessária.** Garante que todos os processos que desejam participar ou executar uma ação terão a chance de fazê-lo em um tempo finito. Evita que alguns nós sejam constantemente ignorados. |
| **Tolerância a Falhas** | O sistema deve continuar mesmo com falhas parciais (ex: falha de um ou mais nós). | **Definitória.** Esta propriedade, juntamente com a **Confiabilidade** e a **Disponibilidade**, é crucial para manter o serviço em funcionamento (alta disponibilidade). |

# 🔄 Algoritmos de Comunicação e Consenso

Algoritmos de Comunicação e Consenso apontam para os dois pilares mais importantes e desafiadores dos sistemas distribuídos:

**Comunicação:** Como os nós (processos/computadores) trocam informações de maneira eficiente, confiável e ordenada, já que não compartilham uma memória central.

**Consenso:** Como esses nós chegam a um acordo sobre um único valor ou estado, mesmo que haja falhas, atrasos ou nós mal-intencionados na rede.

# Algoritmos de Comunicação e Consenso

## Desafios:

| Problema | Descrição | Impacto |
| --- | --- | --- |
| **Latência** | Tempo de atraso na entrega de mensagens. | Pode causar inconsistências temporárias. |
| **Consistência** | Garantir que todos os nós tenham a mesma visão dos dados. | Dificulta o sincronismo em tempo real. |
| **Disponibilidade** | Manter o sistema funcional mesmo se parte dele falhar. | Requer replicação e redundância. |

## 💡 Lei de CAP (Teorema CAP):

Um sistema distribuído **não pode garantir simultaneamente**:

**C**onsistência,
**A**vailability (disponibilidade), e
**P**artition tolerance (tolerância a falhas de rede).

👉 Portanto, os sistemas escolhem **dois** desses três requisitos.

## 💡 Lei de CAP (Teorema CAP):

**Detalhamento das Propriedades:**

1. **C: Consistência (Consistency)**
O que significa: Toda leitura (requisição) deve receber o dado escrito mais recente ou um erro. Em outras palavras, todos os nós devem "ver" os mesmos dados ao mesmo tempo.
2. **A: Altíssima Disponibilidade (Availability)**
O que significa: Toda requisição (leitura ou escrita) para um nó que não falhou deve receber uma resposta (não um erro), garantindo que o sistema esteja sempre operacional.
3. **P: Tolerância à Partição (Partition Tolerance)**
O que significa: O sistema continua operando mesmo que haja uma "partição" na rede. Uma partição ocorre quando a comunicação entre grupos de nós é interrompida (por falha de rede), mas os nós dentro de cada grupo continuam funcionando.

# Algoritmos de Consenso

**Objetivo:**
Fazer com que **todos os nós concordem sobre um mesmo valor ou decisão**, mesmo com falhas ou mensagens fora de ordem.

**Problemas resolvidos por consenso:**

- Eleição de líder.
- Confirmação de transações em blockchain.
- Coordenação de réplicas em bancos de dados.

# Principais Algoritmos:

| Algoritmo | Características | Uso comum |
| --- | --- | --- |
| **Paxos** | Garante consenso mesmo com falhas de nós; complexo, mas muito confiável. | Serviços críticos e sistemas financeiros. |
| **Raft** | Alternativa mais compreensível ao Paxos; define líder e sincroniza logs. | Bancos de dados distribuídos e clusters. |
| **Byzantine Fault Tolerance (BFT)** | Tolera nós maliciosos ou com comportamento arbitrário. | Blockchain e sistemas de alta segurança. |

## Exemplo Prático: Exclusão Mútua Distribuída

**Problema:**
Apenas um processo pode acessar um recurso compartilhado por vez.

**Solução (Ricart-Agrawala):**

- Cada processo envia requisição aos outros.
- Recebe “OK” de todos para entrar na seção crítica.
- Após sair, envia liberação.

# Exemplos Práticos

https://colab.research.google.com/drive/1rQcOr8hoZv_DCLQOzX7JpsaMIpSw66YU?usp=sharing

## 🧩 1 Questões Objetivas:

**1.** Em um sistema distribuído, qual das opções representa corretamente o papel dos algoritmos distribuídos?

a) Centralizar a execução em um único servidor para reduzir falhas.

b) Permitir que vários processos cooperem sem compartilhar memória, trocando mensagens.

c) Aumentar a dependência de um nó mestre.

d) Evitar comunicação entre nós para reduzir latência.

## 🧩 1 Questões Objetivas:

**2.** No contexto do Teorema CAP, é correto afirmar que:

a) Um sistema pode garantir Consistência, Disponibilidade e Tolerância à Partição simultaneamente.

b) O Teorema CAP aplica-se apenas a redes locais.

c) Em caso de partição, o sistema deve escolher entre Consistência e Disponibilidade.

d) A Tolerância à Partição é opcional em sistemas distribuídos modernos.

## 🧩 1 Questões Objetivas:

**3.** Em um algoritmo de eleição de líder (ex.: Bully), o líder é normalmente:

a) O nó que inicia o sistema primeiro.

b) O nó com o menor identificador (ID).

c) O nó com o maior identificador (ID) ativo no sistema.

d) O nó com maior capacidade de armazenamento.

## 🧩 1 Questões Objetivas:

**4.** No algoritmo de exclusão mútua distribuída de Ricart-Agrawala, o acesso à seção crítica é concedido:

a) Após o nó receber permissão de pelo menos um outro processo.

b) Assim que o nó enviar sua solicitação.

c) Quando o nó recebe mensagens “OK” de todos os outros nós.

d) Somente quando o líder permitir explicitamente.

## 🧩 1 Questões Objetivas:

**5.** O algoritmo de consenso em blockchains (como Proof-of-Work) garante que:

a) Apenas um nó armazene todas as transações.

b) Todos os nós concordem sobre o estado do registro, mesmo sem autoridade central.

c) Todas as transações são processadas sequencialmente por um nó mestre.

d) A rede blockchain não precise de validação de blocos.

## 🧩 1 Questões Objetivas:

**6.** A principal diferença entre Raft e Paxos está em:

a) Raft ser mais fácil de entender e implementar, mantendo a mesma segurança teórica de Paxos.

b) Paxos exigir um líder fixo e Raft não.

c) Raft não lidar com falhas de rede.

d) Paxos não garantir consenso em caso de falha.

## 🧠 2 Questões Discursivas

**1.**Explique o que é um algoritmo distribuído e descreva as suas principais propriedades (execução cooperativa, comunicação por mensagens, controle não centralizado).

**2.** O que é consenso em sistemas distribuídos? Por que é um problema difícil de resolver?

**3.** Descreva com suas palavras o significado da **Lei de CAP** e cite exemplos reais de sistemas que priorizam cada uma das propriedades (C, A e P).

## 🧠 2 Questões Discursivas

**4.** Diferencie os algoritmos de comunicação e os algoritmos de consenso. Dê um exemplo de cada.

**5.** Como o algoritmo de **exclusão mútua distribuída** garante que apenas um processo acesse um recurso compartilhado por vez?

**6.** Em um ambiente de IoT com sensores espalhados geograficamente, quais desafios de comunicação e consistência podem ocorrer? Como os algoritmos distribuídos ajudam a resolvê-los?

## 🧠 Estudo de caso real

**7.** Pesquise um sistema real que use algoritmos distribuídos (ex: Blockchain, Cassandra, AWS Lambda).

Explique:

Qual algoritmo de consenso utiliza.
Como lida com falhas e latência.
Quais princípios do Teorema CAP ele prioriza.

# Dúvidas?



cilene.real@fatec.sp.gov.br

# MUITO OBRIGADA!!!!

